Práctica N°1: Inicializacion de OLED en la placa ZedBoard

Índice

1. Objetivos de la práctica **2**
- 1.1. Objetivo general . . . 2
- 1.2. Objetivos específicos . . . 2

2. Reconocimiento de las características de la ZedBoard (Zynq-7000 XC7Z020) **2**
- 2.1. Resumen de la plataforma . . . 2
- 2.2. Asignación de bancos y voltajes . . . 4
- 2.3. Memoria del sistema . . . 5
- 2.4. Interfaces de visualización y audio . . . 5
- 2.5. USB, UART y JTAG . . . 5
- 2.6. Red Ethernet . . . 5
- 2.7. E/S de usuario . . . 5
- 2.8. Conectores de expansión . . . 6
- 2.9. Reloj y reset . . . 6
- 2.10. Modos de configuración/arranque . . . 6
- 2.11. Alimentación y consumo . . . 6
- 2.12. Jumper relevantes (por defecto) . . . 6

3. Arranque desde microSD (ZedBoard) **7**

4. Descripción del codigo **8**
- 4.1. ROM de fuente ASCII (ascii_rom.vhd) . . . 8
- 4.2. Temporizador por milisegundos (delay.vhd) . . . 15
- 4.3. Controlador SPI para OLED (spi$_c$*trl.vhd*) . . . 19
- 4.4. Secuencia de inicialización de la OLED (oled$_i$*nit.vhd*) . . . 23
- 4.5. Demo de escritura en la OLED (oled$_e$*x.vhd*) . . . 31
- 4.6. Módulo tope del controlador OLED (oled ctrl.vhd) . . . 42
- 4.7. Asignación de pines para la OLED y señales de sistema . . . 47

1. Objetivos de la práctica

1.1. Objetivo general

Implementar un sistema completo en la tarjeta ZedBoard para el control de una pantalla OLED mediante descripciones en VHDL, integrando módulos de inicialización, temporización y comunicación SPI, con el fin de comprender el flujo de diseño digital en Vivado y la interacción hardware–software en un entorno FPGA.

1.2. Objetivos específicos

- Reconocer las características técnicas de la ZedBoard relevantes para el manejo de periféricos externos.
- Configurar el arranque de la tarjeta mediante memoria SD, verificando el proceso de carga en la terminal serie.
- Desarrollar y analizar los bloques VHDL de soporte: `ascii_rom`, `delay`, `spi_ctrl`, `oled_init`, `oled_ex` y `oled_ctrl`.
- Asignar correctamente los pines físicos del FPGA para garantizar la comunicación con la pantalla OLED en los niveles de tensión adecuados.
- Validar el funcionamiento del sistema en placa mostrando el alfabeto y el mensaje *"Hello World!"* en la pantalla OLED.

2. Reconocimiento de las características de la ZedBoard (Zynq-7000 XC7Z020)

2.1. Resumen de la plataforma

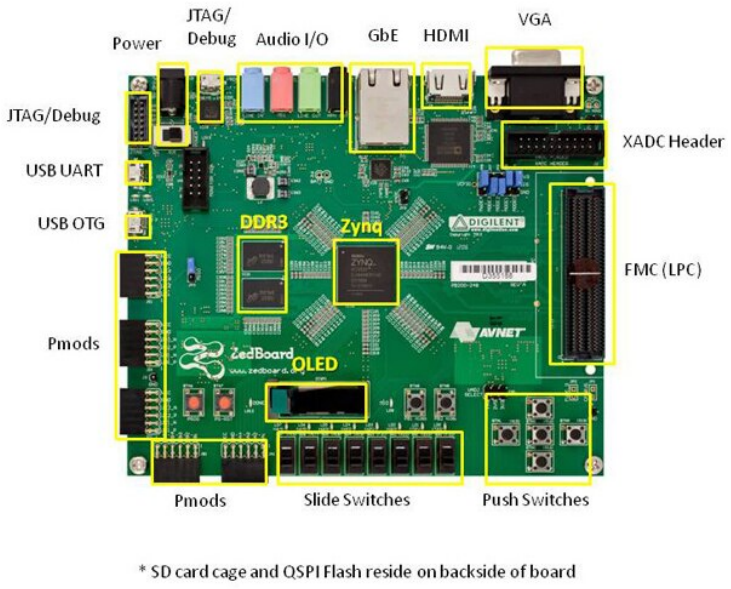
La ZedBoard es una tarjeta de evaluación basada en el SoC Xilinx Zynq–7000 **XC7Z020-1CLG484C**, que integra un *Processing System* (PS) con doble *ARM Cortex-A9* y lógica programable *7-Series* ($m85\,\mathrm{k}$ celdas lógicas), más un conjunto robusto de periféricos y cabezales de expansión.

- **SoC**: Zynq XC7Z020 (PS + PL). Configuración primaria desde *QSPI Flash*. Alternativas: *JTAG* en cascada e *SD Card*.
- **Memoria a bordo**: 512MB DDR3 (128M×16 ×2, bus de 32 bits), 256Mb *Quad-SPI* Flash.

- **Display/Audio**: HDMI (ADV7511), VGA 12-bit (R,G,B por R-2R), OLED 128×32 (UG-2832HSWEG04), códec I^2S ADAU1761 (entrada/-salida estéreo, 8–96 kHz).
- **USB**: USB-JTAG integrado (SMT1-equivalente), USB-UART (CY7C64225), USB OTG (TUSB1210, *ver nota de compatibilidad*).
- **Red**: PHY Gigabit Ethernet 88E1518 vía RGMII, con RJ45 de magnetismo integrado.
- **Expansión**: 1x FMC LPC (Vadj seleccionable), 5x Pmod (4 para PL, 1 para PS/MIO), con protección ESD en líneas de datos.
- **Relojes**: 33.333MHz (PS) y 100MHz (PL).
- **E/S de usuario**: 7 botones (5 PL, 2 PS), 8 *slide switches* (PL), 9 LEDs de usuario (8 PL, 1 PS), LED DONE (PL).
- **Alimentación**: Entrada 12V @ 5A (jack 5.5/2.5 mm, centro positivo), con reguladores y secuenciación acorde a Zynq; LED *Power Good*.

Nota crítica (USB OTG). Tras el diseño, se reportó una incompatibilidad temporal de temporización entre el PHY TUSB1210 y Zynq; no se recomienda en diseños nuevos—revisar errata del fabricante para detalles y mitigaciones.

Referencia Visual de la placa. De manera adicional se incluye una imagen referencial del dispositivo en el que se enlistan los distintos puertos disponibles.



Placa de desarrollo ZedBoard (Zynq-7000 XC7Z020)

2.2. Asignación de bancos y voltajes

El encapsulado CLG484 organiza los bancos PS/PL según la Fig. **??** (ver guía). Los voltajes por banco en ZedBoard son los de la Tabla 1.

Cuadro 1: Voltajes por banco (ZedBoard).

Sección	**Banco**	**Voltaje (por defecto)**
PS (MIO)	MIO 0/500	3.3 V
	MIO 1/501	1.8 V
DDR	DDR	1.5 V
PL	Bank 0	3.3 V
	Bank 13	3.3 V
	Bank 33	3.3 V
	Bank 34	Vadj (2.5 V por defecto)
	Bank 35	Vadj (2.5 V por defecto)

Advertencia Vadj (FMC). *Jumper* J18 selecciona Vadj (1.8/2.5/3.3 V). Por seguridad, 3.3 V no viene poblado para no dañar *mezzanines* FMC que sólo admiten 1.8/2.5 V. Verifica **siempre** J18 antes de conectar una tarjeta FMC.

2.3. Memoria del sistema

DDR3: 2× Micron 128 M×16 ($x32$ total) = 512MB, con controlador/PHY duros en PS. Velocidades hasta 533MHz (1066 MT/s). Tensión 1.5 V, terminación VTT 0.75 V, VREF=0.5×VDDQ.

QSPI Flash: Spansion S25FL256S 256Mb, x1/x2/x4, hasta 104MHz; interfaz por MIO[1:6,8] (feedback en MIO[8]) y *primary boot*.

SD: Interfaz SD0 PS por MIO[40–47]; traslación de nivel vía TXS02612 (la ranura es 3.3 V, MIO 1.8 V).

2.4. Interfaces de visualización y audio

HDMI: ADV7511 (225 MHz, HDMI 1.4/DVI 1.0, hasta 1080p60, YCbCr 4:2:2 de 16 bit).

VGA: 12 bits (4 bits por color desde PL mediante red resistiva).

OLED: UG-2832HSWEG04 128×32 mono; interfaz SPI (CS#, RES#, D/C#, SCLK, SDIN) y pines de potencia/booster (VBAT, VCC, etc.).

Audio: ADAU1761 (I^2S), 8–96 kHz, control por I^2C, *line/mic in*, *line/headphone out*.

2.5. USB, UART y JTAG

USB OTG: TUSB1210 por ULPI → MIO[28–39]; ver nota de compatibilidad.

USB–UART: CY7C64225 con traslación TXS0102; PS UART1 por MIO[48–49] (1.8 V ↔ 3.3 V).

USB–JTAG: Módulo Digilent SMT1–equivalente, plenamente soportado por Vivado/SDK.

2.6. Red Ethernet

PHY Gigabit 88E1518 (RGMII sobre MIO 1/501 a 1.8 V, con PS_MIO_VREF=0.9 V). Conector RJ45 con magnetismo y LEDs de estado.

2.7. E/S de usuario

Botones: 5 PL (BTNU/BTND/BTNL/BTNR/BTNC), 2 PS (PB1/PB2, MIO50/51).

Switches: 8 *slide switches* (SW0–SW7) hacia PL, polarizados mediante resistores a Vadj/GND.

LEDs: 8 PL (LD0–LD7, activos en alto, limitación 390 Ω), 1 PS (LD9, MIO7).

2.8. Conectores de expansión

FMC LPC: Bancos 34/35 con Vadj seleccionable (1.8/2.5/3.3 V); CC/MRCC según guía, restricciones de *skew* e impedancia; **verifica J18** antes de acoplar.

Pmod (2x6): 5 conectores (4 en PL, 1 PS/MIO[0,9–15]); pares diferenciales en JC1/JD1 para LVDS hasta $m525$ Mb/s; ESD en líneas de datos.

AMS (XADC): Cabecera analógica (VP/VN, VAUX0/8, DXP/DXN), filtros anti-*aliasing*, VREF 1.25 V y VCCADC 1.8 V.

2.9. Reloj y reset

Relojes: 33.333MHz (PS) y 100MHz (PL, GC pin en Bank 13).

Resets: *PS_POR_B* (monitorea regulación de fuentes), *PROG_B* (BTN6, reinicia PL), *PS_RST/BTN7* (reset del sistema sin re-muestrear modo de arranque).

2.10. Modos de configuración/arranque

Selección por *mode pins* MIO[6:2]: **SD Card** por defecto (JP8–JP10 = 110), *QSPI*, *JTAG*; JP11 decide uso de PLL internos. MIO[2] define *JTAG cascaded* (0) o independiente (1).

2.11. Alimentación y consumo

Entrada primaria 12V (jack 5.5/2.5 mm, 4A–5A); protección, *current shunt* 10mΩ (medición en J21), secuenciación conforme a Zynq (Vccint → Vccaux → Vcco/Vadj → DDR/VTT), LED *Power Good*. Tabla de corrientes estimadas disponible en la guía (p.ej., PS/PL, DDR, periféricos).

2.12. Jumper relevantes (por defecto)

- **JP6**: Pull-down en PS_MIO0 (requerido para boot SD en silicio CES).
- **JP7–JP11**: *Boot Mode* y PLL (SD por defecto; PLL usado).
- **J18**: Selección Vadj (1.8 V por defecto).

Consulta el mapa de jumpers de la placa para posiciones exactas.

3. Arranque desde microSD (ZedBoard)

1) ¿Cuál es el modo de arrancar la placa usando una SD?

1. **La ZedBoard arranca por defecto desde microSD** (el PS lee los *mode pins* MIO[8:2] y selecciona SD como dispositivo primario).

2. Inserta una microSD (Clase 4+). La interfaz SD0 del PS usa MIO[40–47] y un traductor TXS02612 (3.3 V ↔ 1.8 V), con **JP6 en corto para habilitar SD**.

3. Formatea en **FAT32** y coloca en la raíz los archivos de arranque (`BOOT.BIN` y, si usas Linux, `boot.scr`/`image.ub`). La SD sirve tanto para *boot* como para almacenamiento.

4. Enciende la placa. (Si tu diseño requiere configurar la PL al inicio, incluye el `.bit` dentro de `BOOT.BIN` para que el FSBL lo cargue).

2) ¿Cómo se ve el proceso en la terminal COM?

1. Conecta el **USB–UART** en **J14**. Este puente (CY7C64225) expone el **UART1 del PS** por MIO[48:49] mediante un *level shifter* TXS0102.

2. Abre un terminal serial (p. ej., TeraTerm/Putty) en el puerto COM enumerado. Verás mensajes del **FSBL/U-Boot/Linux** durante el *boot*. (La velocidad típica de imágenes de referencia es 115200 8N1; usa la que defina tu *build*).

3) ¿Cómo cargar otra imagen Linux (cambiar el programa de la SD)?

1. Apaga la placa y retira la microSD.

2. En la partición **FAT32**, reemplaza los ficheros según tu *flow*:

   - **Mínimo**: `BOOT.BIN` (FSBL + U-Boot; opcionalmente el `.bit` si quieres configurar PL al inicio).
   - **Arranque directo**: `boot.scr` (script de U-Boot) y `image.ub` (kernel+dtb+initramfs) u `Image`/`system.dtb`/rootfs.

3. Inserta la SD y enciende. El PS **bootea desde SD** y ejecuta lo nuevo sin usar JTAG.

4. Descripción del codigo

4.1. ROM de fuente ASCII (ascii_rom.vhd)

Código fuente

```
-- Description: ROM containing ASCII character data.

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity ascii_rom is
    port (  clk     : in std_logic; -- System clock
            addr    : in std_logic_vector (10 downto 0);
                -- Memory address
            dout    : out std_logic_vector (7 downto 0))
               ; -- Data from memory
end ascii_rom;

architecture behavioral of ascii_rom is

    -- 1024 elements, 8 bits each
    type memory is array (0 to 1023) of std_logic_vector
        (7 downto 0);

    -- ASCII character data
    constant rom_data : memory := ( x"00", x"00", x"00",
        x"00", x"00", x"00", x"00", x"00",
    x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
       00",
    x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
       00",
    x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
       00",
    x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
       00",
    x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
       00",
    x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
       00",
```

```
27      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
28      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
29      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
30      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
31      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
32      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
33      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
34      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
35      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
36      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
37      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
38      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
39      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
40      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
41      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
42      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
43      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
44      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
45      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
46      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
47      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
```

```
48      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
49      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
50      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
51      x"00", x"00", x"00", x"00", x"00", x"00", x"00", x"
            00",
52      x"00", x"00", x"00", x"5f", x"00", x"00", x"00", x"
            00",
53      x"00", x"00", x"03", x"00", x"03", x"00", x"00", x"
            00",
54      x"64", x"3c", x"26", x"64", x"3c", x"26", x"24", x"
            00",
55      x"26", x"49", x"49", x"7f", x"49", x"49", x"32", x"
            00",
56      x"42", x"25", x"12", x"08", x"24", x"52", x"21", x"
            00",
57      x"20", x"50", x"4e", x"55", x"22", x"58", x"28", x"
            00",
58      x"00", x"00", x"00", x"03", x"00", x"00", x"00", x"
            00",
59      x"00", x"00", x"1c", x"22", x"41", x"00", x"00", x"
            00",
60      x"00", x"00", x"00", x"41", x"22", x"1c", x"00", x"
            00",
61      x"00", x"15", x"15", x"0e", x"0e", x"15", x"15", x"
            00",
62      x"00", x"08", x"08", x"3e", x"08", x"08", x"00", x"
            00",
63      x"00", x"00", x"00", x"50", x"30", x"00", x"00", x"
            00",
64      x"00", x"08", x"08", x"08", x"08", x"08", x"00", x"
            00",
65      x"00", x"00", x"00", x"40", x"00", x"00", x"00", x"
            00",
66      x"40", x"20", x"10", x"08", x"04", x"02", x"01", x"
            00",
67      x"00", x"3e", x"41", x"41", x"41", x"3e", x"00", x"
            00",
68      x"00", x"00", x"41", x"7f", x"40", x"00", x"00", x"
            00",
```

```
69      x"00", x"42", x"61", x"51", x"49", x"6e", x"00", x"
           00",
70      x"00", x"22", x"41", x"49", x"49", x"36", x"00", x"
           00",
71      x"00", x"18", x"14", x"12", x"7f", x"10", x"00", x"
           00",
72      x"00", x"27", x"49", x"49", x"49", x"71", x"00", x"
           00",
73      x"00", x"3c", x"4a", x"49", x"48", x"70", x"00", x"
           00",
74      x"00", x"43", x"21", x"11", x"0d", x"03", x"00", x"
           00",
75      x"00", x"36", x"49", x"49", x"49", x"36", x"00", x"
           00",
76      x"00", x"06", x"09", x"49", x"29", x"1e", x"00", x"
           00",
77      x"00", x"00", x"00", x"12", x"00", x"00", x"00", x"
           00",
78      x"00", x"00", x"00", x"52", x"30", x"00", x"00", x"
           00",
79      x"00", x"00", x"08", x"14", x"14", x"22", x"00", x"
           00",
80      x"00", x"14", x"14", x"14", x"14", x"14", x"14", x"
           00",
81      x"00", x"00", x"22", x"14", x"14", x"08", x"00", x"
           00",
82      x"00", x"02", x"01", x"59", x"05", x"02", x"00", x"
           00",
83      x"3e", x"41", x"5d", x"55", x"4d", x"51", x"2e", x"
           00",
84      x"40", x"7c", x"4a", x"09", x"4a", x"7c", x"40", x"
           00",
85      x"41", x"7f", x"49", x"49", x"49", x"49", x"36", x"
           00",
86      x"1c", x"22", x"41", x"41", x"41", x"41", x"22", x"
           00",
87      x"41", x"7f", x"41", x"41", x"41", x"22", x"1c", x"
           00",
88      x"41", x"7f", x"49", x"49", x"5d", x"41", x"63", x"
           00",
89      x"41", x"7f", x"49", x"09", x"1d", x"01", x"03", x"
           00",
```

```
90      x"1c", x"22", x"41", x"49", x"49", x"3a", x"08", x"
            00",
91      x"41", x"7f", x"08", x"08", x"08", x"7f", x"41", x"
            00",
92      x"00", x"41", x"41", x"7F", x"41", x"41", x"00", x"
            00",
93      x"30", x"40", x"41", x"41", x"3F", x"01", x"01", x"
            00",
94      x"41", x"7f", x"08", x"0c", x"12", x"61", x"41", x"
            00",
95      x"41", x"7f", x"41", x"40", x"40", x"40", x"60", x"
            00",
96      x"41", x"7f", x"42", x"0c", x"42", x"7f", x"41", x"
            00",
97      x"41", x"7f", x"42", x"0c", x"11", x"7f", x"01", x"
            00",
98      x"1c", x"22", x"41", x"41", x"41", x"22", x"1c", x"
            00",
99      x"41", x"7f", x"49", x"09", x"09", x"09", x"06", x"
            00",
100     x"0c", x"12", x"21", x"21", x"61", x"52", x"4c", x"
            00",
101     x"41", x"7f", x"09", x"09", x"19", x"69", x"46", x"
            00",
102     x"66", x"49", x"49", x"49", x"49", x"49", x"33", x"
            00",
103     x"03", x"01", x"41", x"7f", x"41", x"01", x"03", x"
            00",
104     x"01", x"3f", x"41", x"40", x"41", x"3f", x"01", x"
            00",
105     x"01", x"0f", x"31", x"40", x"31", x"0f", x"01", x"
            00",
106     x"01", x"1f", x"61", x"14", x"61", x"1f", x"01", x"
            00",
107     x"41", x"41", x"36", x"08", x"36", x"41", x"41", x"
            00",
108     x"01", x"03", x"44", x"78", x"44", x"03", x"01", x"
            00",
109     x"43", x"61", x"51", x"49", x"45", x"43", x"61", x"
            00",
110     x"00", x"00", x"7f", x"41", x"41", x"00", x"00", x"
            00",
```

```
111     x"01", x"02", x"04", x"08", x"10", x"20", x"40", x"
            00",
112     x"00", x"00", x"41", x"41", x"7f", x"00", x"00", x"
            00",
113     x"00", x"04", x"02", x"01", x"01", x"02", x"04", x"
            00",
114     x"00", x"40", x"40", x"40", x"40", x"40", x"40", x"
            00",
115     x"00", x"01", x"02", x"00", x"00", x"00", x"00", x"
            00",
116     x"00", x"34", x"4a", x"4a", x"4a", x"3c", x"40", x"
            00",
117     x"00", x"41", x"3f", x"48", x"48", x"48", x"30", x"
            00",
118     x"00", x"3c", x"42", x"42", x"42", x"24", x"00", x"
            00",
119     x"00", x"30", x"48", x"48", x"49", x"3f", x"40", x"
            00",
120     x"00", x"3c", x"4a", x"4a", x"4a", x"2c", x"00", x"
            00",
121     x"00", x"00", x"48", x"7e", x"49", x"09", x"00", x"
            00",
122     x"00", x"26", x"49", x"49", x"49", x"3f", x"01", x"
            00",
123     x"41", x"7f", x"48", x"04", x"44", x"78", x"40", x"
            00",
124     x"00", x"00", x"44", x"7d", x"40", x"00", x"00", x"
            00",
125     x"00", x"00", x"40", x"44", x"3d", x"00", x"00", x"
            00",
126     x"41", x"7f", x"10", x"18", x"24", x"42", x"42", x"
            00",
127     x"00", x"40", x"41", x"7f", x"40", x"40", x"00", x"
            00",
128     x"42", x"7e", x"02", x"7c", x"02", x"7e", x"40", x"
            00",
129     x"42", x"7e", x"44", x"02", x"42", x"7c", x"40", x"
            00",
130     x"00", x"3c", x"42", x"42", x"42", x"3c", x"00", x"
            00",
131     x"00", x"41", x"7f", x"49", x"09", x"09", x"06", x"
            00",
```

```
132     x"00", x"06", x"09", x"09", x"49", x"7f", x"41", x"
            00",
133     x"00", x"42", x"7e", x"44", x"02", x"02", x"04", x"
            00",
134     x"00", x"64", x"4a", x"4a", x"4a", x"36", x"00", x"
            00",
135     x"00", x"04", x"3f", x"44", x"44", x"20", x"00", x"
            00",
136     x"00", x"02", x"3e", x"40", x"40", x"22", x"7e", x"
            40",
137     x"02", x"0e", x"32", x"40", x"32", x"0e", x"02", x"
            00",
138     x"02", x"1e", x"62", x"18", x"62", x"1e", x"02", x"
            00",
139     x"42", x"62", x"14", x"08", x"14", x"62", x"42", x"
            00",
140     x"01", x"43", x"45", x"38", x"05", x"03", x"01", x"
            00",
141     x"00", x"46", x"62", x"52", x"4a", x"46", x"62", x"
            00",
142     x"00", x"00", x"08", x"36", x"41", x"00", x"00", x"
            00",
143     x"00", x"00", x"00", x"7f", x"00", x"00", x"00", x"
            00",
144     x"00", x"00", x"00", x"41", x"36", x"08", x"00", x"
            00",
145     x"00", x"18", x"08", x"08", x"10", x"10", x"18", x"
            00",
146     x"AA", x"55", x"AA", x"55", x"AA", x"55", x"AA", x"
            55");
147
148 begin
149
150     process (clk)
151     begin
152         if rising_edge(clk) then
153             dout <= rom_data(conv_integer(addr));
154         end if;
155     end process;
156
157 end behavioral;
```

Listing 1: ascii_rom.vhd

Lectura guiada y consideraciones de integración

- **Interfaz**:
  - `clk`: lectura *síncrona*; la salida `dout` se actualiza en flanco ascendente.
  - `addr(10 downto 0)`: dirección de 11 bits. La ROM tiene 1024 ubicaciones (0..1023); por tanto, **usa sólo 10 bits efectivos** o garantiza que `addr(10)` siempre sea '0' para evitar accesos fuera de rango.
  - `dout(7 downto 0)`: byte de patrón de columna/fila del glifo (1 bpp).
- **Organización de memoria**: $1024 \times 8 = 8192$ bits ($\approx$8 Kb). En 7-Series suele inferir 1 bloque `RAMB18E1` (o LUT ROM si el sintetizador decide). Con salida registrada, se cumple un timing cómodo para relojes de sistema comunes (50–100 MHz).
- **Estilo y librerías**:
  - Se usando `std_logic_arith` y `std_logic_unsigned` + `conv_integer`. Son extensiones no estándar pero muy usadas.
  - `conv_integer(addr)` asume que `addr` es un vector sin 'X/Z';
- **Mapeo típico de glifos** :

$$\text{addr} = (\text{código_ASCII}) \times 8 \; + \; \text{fila_de_columna}$$

con fila_de_columna $\in [0, 7]$ si el glifo es 8×8. Se debe ajustar si la fuente es 5×8 u otra variante. Ademas se aconseja verificar también si el bit menos significativo representa píxel "arriba" o "abajo".

4.2. Temporizador por milisegundos (delay.vhd)

Código fuente

```
1
2 -- Description: Creates a delay of delay_ms ms.
3 --
4
5 library ieee;
6 use ieee.std_logic_1164.all;
7 use ieee.std_logic_arith.all;
```

```
8  use ieee.std_logic_unsigned.all;
9
10 entity delay is
11     port (  clk          : in std_logic; -- System clock
12             rst          : in std_logic;  -- Global
                   synchronous reset
13             delay_ms     : in std_logic_vector (11 downto
                    0); -- Amount of ms to delay
14             delay_en     : in std_logic; -- Delay enable
15             delay_fin    : out std_logic); -- Delay
                   finish flag
16 end delay;
17
18 architecture behavioral of delay is
19
20     type states is (Idle, Hold, Done);
21
22     signal current_state : states := Idle; -- Signal for
           state machine
23     signal clk_counter : std_logic_vector(16 downto 0)
          := (others => '0'); -- Counts up on every rising
          edge of clk
24     signal ms_counter : std_logic_vector (11 downto 0)
          := (others => '0'); -- Counts up when clk_counter
           = 100,000
25
26 begin
27     -- delay_fin goes high when delay is done
28     delay_fin <= '1' when (current_state = Done and
          delay_en = '1') else '0';
29
30     -- State machine for Delay block
31     state_machine : process (clk)
32     begin
33         if rising_edge(clk) then
34             if rst = '1' then -- When rst is asserted
                   switch to Idle (synchronous)
35                 current_state <= Idle;
36             else
37                 case current_state is
38                     when Idle =>
39                         if delay_en = '1' then -- Start
                               delay on delay_en
```

```
40                              current_state <= Hold;
41                          end if;
42                      when Hold =>
43                          if ms_counter = delay_ms then --
                                Stay until delay_ms has
                               occured
44                              current_state <= Done;
45                          end if;
46                      when Done =>
47                          if delay_en = '0' then -- Wait
                               until delay_en is deasserted
                               to go to Idle
48                              current_state <= Idle;
49                          end if;
50                      when others =>
51                          current_state <= Idle;
52                  end case;
53              end if;
54          end if;
55      end process state_machine;
56      -- Creates ms_counter that counts at 1KHz
57      clk_div : process (clk)
58      begin
59          if rising_edge(clk) then
60              if current_state = Hold then
61                  if clk_counter = "11000011010100000"
                       then -- 100,000
62                      clk_counter <= (others => '0');
63                      ms_counter <= ms_counter + 1; --
                           Increments at 1KHz
64                  else
65                      clk_counter <= clk_counter + 1;
66                  end if;
67              else -- If not in the hold state reset
                   counters
68                  clk_counter <= (others => '0');
69                  ms_counter <= (others => '0');
70              end if;
71          end if;
72      end process;
73
74  end behavioral;
```

Listing 2: delay.vhd

Lectura guiada y criterios de integración

- **Propósito**. Se implementa un retardo programable en milisegundos mediante una FSM con tres estados: *Idle* (espera), *Hold* (conteo) y *Done* (finalización mediante `delay_fin`).
- **Suposición de reloj**. El umbral `100000` implica 100MHz de `clk` (1ms = 100 000 ciclos). Si `clk` difiere, se debe ajustar el umbral o parametrizarlo.
- **Rango**. `delay_ms` y `ms_counter` son de 12 bits ⇒ hasta 4095 ms (≈4.095 s).
- **Bandera de fin**. `delay_fin` permanece en '1' mientras el estado sea *Done* y `delay_en` siga en '1'. Cuando `delay_en` cae a '0', el módulo vuelve a *Idle*.
- **Entradas dinámicas**. Si `delay_ms` cambia durante *Hold*, el objetivo se mueve; para comportamiento determinista se debe capturar `delay_ms` al entrar en *Hold*.
- **Sincronización**. `delay_en` debe estar sincronizada a `clk`; si proviene de otro dominio o de un botón, se deben usar dos FF de sincronía y, si aplica, antirrebote.
- **Anchos de contador**. `clk_counter` (17 bits) cubre 100 000 correctamente ($2^{17} = 131{,}072$).
- **Uso típico con OLED**. En secuencias de *power-on* y comandos sensibles, se debe: programar `delay_ms` → poner `delay_en`=1 → esperar `delay_fin` → avanzar de estado.

4.3. Controlador SPI para OLED ($spi_ctrl.vhd$)

Código fuente

```
1  --
2  -- Written by Ryan Kim, Digilent Inc.
3  -- Modified by Michael Mattioli
4  --
5  -- Description: SPI block that sends SPI data formatted
       oled_sclk active low with sdout changing
6  -- on the falling edge.
7  --
8
9  library ieee;
10 use ieee.std_logic_1164.all;
11 use ieee.std_logic_arith.all;
12 use ieee.std_logic_unsigned.all;
13
14 entity spi_ctrl is
15     Port ( clk          : in std_logic; -- System clok
           (100MHz)
16            rst          : in std_logic; -- Global
                 synchronous reset
17            en           : in std_logic; -- Block enable
                 pin
18            sdata        : in std_logic_vector (7 downto
                 0); -- Byte to be sent
19            sdout        : out std_logic; -- Serial data
                 out
20            oled_sclk    : out std_logic; -- OLED serial
                 clock
21            fin          : out std_logic); -- Finish flag
                  for block
22 end spi_ctrl;
23
24 architecture behavioral of spi_ctrl is
25
26     type states is (Idle, Send, Done);
27
28     signal current_state : states := Idle; -- Signal for
           state machine
29
30     signal shift_register   : std_logic_vector (7 downto
           0); -- Shift register to shift out sdata saved
```

```
        when en was set
31      signal shift_counter    : std_logic_vector (3 downto
            0); -- Keeps track how many bits were sent
32      signal clk_divided      : std_logic := '1'; -- Used
            as oled_sclk
33      signal counter          : std_logic_vector (4 downto
            0) := (others => '0'); -- Count clocks to be
            used to divide clk
34      signal temp_sdata       : std_logic := '1'; -- Tied
            to sdout
35
36      signal falling : std_logic := '0'; -- Signal
            indicating that the clk has just fell
37
38  begin
39
40      clk_divided <= not counter(4); -- oled_sclk = clk /
            32
41      oled_sclk <= clk_divided;
42      sdout <= temp_sdata;
43      fin <= '1' when current_state = Done else '0';
44
45      state_machine : process (clk)
46      begin
47          if rising_edge(clk) then
48              if rst = '1' then -- Synchronous rst
49                  current_state <= Idle;
50              else
51                  case current_state is
52                      when Idle => -- Wait for en to go
                            high
53                          if en = '1' then
54                              current_state <= Send;
55                          end if;
56                      when Send => -- Start sending bits,
                            transition out when all bits are
                            sent and oled_sclk is high
57                          if shift_counter = "1000" and
                               falling = '0' then
58                              current_state <= Done;
59                          end if;
60                      when Done => -- Finish SPI
                            transimission wait for en to go
```

```
                        low
61                      if en = '0' then
62                          current_state <= Idle;
63                      end if;
64                  when others =>
65                      current_state <= Idle;
66              end case;
67          end if;
68      end if;
69  end process state_machine;
70
71  clk_div : process (clk)
72  begin
73      if rising_edge(clk) then
74          if current_state = Send then -- Start clock
                counter when in send state
75              counter <= counter + 1;
76          else -- Reset clock counter when not in send
                state
77              counter <= (others => '0');
78          end if;
79      end if;
80  end process clk_div;
81
82  spi_send_byte : process (clk) -- Sends SPI data
        formatted oled_sclk active low with sdout
        changing on the falling edge
83  begin
84      if rising_edge(clk) then
85          if current_state = Idle then
86              shift_counter <= (others => '0');
87              shift_register <= sdata; -- Keeps
                    placing sdata into shift_register so
                    that when state goes to send it has
                    the latest sdata
88              temp_sdata <= '1';
89          elsif current_state = Send then
90              if clk_divided = '0' and falling = '0'
                    then -- If on the falling edge of
                    clk_divided
91                  falling <= '1'; -- Indicate that it
                        is passed the falling edge
```

```
92                      temp_sdata <= shift_register(7); --
                           Send out the MSB
93                      shift_register <= shift_register(6
                           downto 0) & '0'; -- Shift through
                            sdata
94                      shift_counter <= shift_counter + 1;
                           -- Keep track of what bit it is
                           on
95                  elsif clk_divided = '1' then -- On
                       oled_sclk high reset the falling flag
96                      falling <= '0';
97                  end if;
98              end if;
99          end if;
100     end process spi_send_byte;
101
102 end behavioral;
```

Listing 3: spi_ctrl.vhd

Lectura guiada y criterios de integración

- **Función y modo SPI**. El bloque envía un byte por SPI con **datos cambiando en el flanco de bajada** de `oled_sclk` y, por tanto, se debe muestrear en el **flanco de subida**. Esto corresponde al modo habitual **CPOL=1, CPHA=1** (SPI modo 3) cuando la línea de reloj permanece en '1' en reposo y las transiciones válidas ocurren alta→baja para el cambio y baja→alta para el muestreo.

- **Frecuencia de reloj SPI**. `oled_sclk` se obtiene como `clk`/32 usando `counter(4)`. Con `clk` = 100 MHz, el reloj SPI nominal es 100 MHz/32 = 3,125 MHz. Si se requiere otra velocidad, se debe ajustar el divisor (ancho del contador o bit usado).

- **Protocolo de envío**. En estado *Send*, el MSB (`shift_register(7)`) se presenta en `sdout` justo en cada flanco de bajada de `oled_sclk` (`clk_divided='0'` con `falling='0'`), luego se desplaza el registro y se incrementa `shift_counter`. Tras 8 bits (`"1000"`) y con `oled_sclk` en alto (`falling='0'`), se pasa a *Done*.

- **Señales de control**. `en` inicia la transacción; `fin` es de nivel alto en *Done* hasta que `en` cae a '0'. Si se requiere un pulso de "fin", se debe derivar un *one-shot* sobre la transición *Send*→*Done*.

- **Cargas e inactividad**. `sdout` queda en '1' fuera de *Send*; `clk_divided` arranca en '1'. Si el periférico OLED exige `SCLK` inactivo alto y `SDOUT` en '1', esta convención es compatible.
- **Interfaz superior**. La gestión de `CS_n` y `DC` no está incluida en este bloque; se debe controlar desde la FSM superior (driver OLED) con temporización coherente: `CS_n` activo antes del primer bit y liberado tras *Done*, `DC` estable durante todo el byte.
- **Dominios y robustez**. `en` y `sdata` deben estar estables y sincronizados a `clk` al entrar en *Send*. Si `sdata` puede cambiar, se debe amostrar en *Idle* (ya contemplado por `shift_register<=sdata`).
- **Casos límite**.
  - Si `en` cae antes de completar 8 bits, la FSM no aborta; finalizará y esperará `en`='0' en *Done*. La lógica superior debe mantener `en`='1' durante toda la transferencia.
  - Si se encadenan bytes, se debe mantener `en`='1', monitorizar `fin` y cargar un nuevo `sdata` durante la ventana de `oled_sclk` alto antes del siguiente flanco de bajada.

4.4. Secuencia de inicialización de la OLED (oled$_{i}nit.vhd$)

Código fuente

```
1  --
2  -- Written by Ryan Kim, Digilent Inc.
3  -- Modified by Michael Mattioli
4  --
5  -- Description: Runs the initialization sequence for the
       OLED display.
6  --
7
8  library ieee;
9  use ieee.std_logic_1164.all;
10 use ieee.std_logic_arith.all;
11 use ieee.std_logic_unsigned.all;
12
13 entity oled_init is
14     port (  clk         : in std_logic; -- System clock
15             rst         : in std_logic; -- Global
                   synchronous reset
```

```
16              en          : in std_logic; -- Block enable
                   pin
17              sdout       : out std_logic; -- SPI data out
18              oled_sclk   : out std_logic; -- SPI clock
19              oled_dc     : out std_logic; -- Data/Command
                    pin
20              oled_res    : out std_logic; -- OLED reset
21              oled_vbat   : out std_logic; -- oled_vbat
                   enable
22              oled_vdd    : out std_logic; -- oled_vdd
                   enable
23              fin         : out std_logic); -- Finish flag
                    for block
24  end oled_init;
25
26  architecture behavioral of oled_init is
27
28      component spi_ctrl
29          port (  clk         : in std_logic;
30                  rst         : in std_logic;
31                  en          : in std_logic;
32                  sdata       : in std_logic_vector (7
                       downto 0);
33                  sdout       : out std_logic;
34                  oled_sclk   : out std_logic;
35                  fin         : out std_logic);
36      end component;
37
38      component delay
39          port (  clk         : in std_logic;
40                  rst         : in std_logic;
41                  delay_ms    : in std_logic_vector (11
                       downto 0);
42                  delay_en    : in std_logic;
43                  delay_fin   : out std_logic);
44      end component;
45
46      type states is (Transition1,
47                      Transition2,
48                      Transition3,
49                      Transition4,
50                      Transition5,
51                      Idle,
```

```
52                      VddOn,
53                      Wait1,
54                      DispOff,
55                      ResetOn,
56                      Wait2,
57                      ResetOff,
58                      ChargePump1,
59                      ChargePump2,
60                      PreCharge1,
61                      PreCharge2,
62                      VbatOn,
63                      Wait3,
64                      DispContrast1,
65                      DispContrast2,
66                      InvertDisp1,
67                      InvertDisp2,
68                      ComConfig1,
69                      ComConfig2,
70                      DispOn,
71                      FullDisp,
72                      Done);
73
74      signal current_state : states := Idle;
75      signal after_state : states := Idle;
76
77      signal temp_dc       : std_logic := '0';
78      signal temp_res      : std_logic := '1';
79      signal temp_vbat     : std_logic := '1';
80      signal temp_vdd      : std_logic := '1';
81      signal temp_fin      : std_logic := '0';
82
83      signal temp_delay_ms     : std_logic_vector (11
           downto 0) := (others => '0');
84      signal temp_delay_en     : std_logic := '0';
85      signal temp_delay_fin    : std_logic;
86      signal temp_spi_en       : std_logic := '0';
87      signal temp_sdata        : std_logic_vector (7 downto
            0) := (others => '0');
88      signal temp_spi_fin      : std_logic;
89
90  begin
91
92      spi_comp: spi_ctrl port map (   clk => clk,
```

```
93                                      rst => rst,
94                                      en => temp_spi_en,
95                                      sdata => temp_sdata,
96                                      sdout => sdout,
97                                      oled_sclk =>
                                           oled_sclk,
98                                      fin => temp_spi_fin)
                                           ;
99
100     delay_comp: delay port map (clk => clk,
101                                 rst => rst,
102                                 delay_ms =>
                                       temp_delay_ms,
103                                 delay_en =>
                                       temp_delay_en,
104                                 delay_fin =>
                                       temp_delay_fin);
105
106     oled_dc <= temp_dc;
107     oled_res <= temp_res;
108     oled_vbat <= temp_vbat;
109     oled_vdd <= temp_vdd;
110     fin <= temp_fin;
111
112     -- Delay 100 ms after VbatOn
113     temp_delay_ms <=    "000001100100" when after_state
           = DispContrast1 else -- 100ms
114                         "000000000001"; -- 1ms
115
116     process (clk)
117     begin
118         if rising_edge(clk) then
119             if rst = '1' then
120                 current_state <= Idle;
121                 temp_res <= '0';
122             else
123                 temp_res <= '1';
124                 case current_state is
125                     when Idle =>
126                         if en = '1' then
127                             temp_dc <= '0';
128                             current_state <= VddOn;
129                         end if;
```

```
130
131                     -- Initialization Sequence
132                     -- This should be done everytime the
                            OLED display is started
133                     when VddOn =>
134                         temp_vdd <= '0';
135                         current_state <= Wait1;
136                     when Wait1 =>
137                         after_state <= DispOff;
138                         current_state <= Transition3;
139                     when DispOff =>
140                         temp_sdata <= "10101110"; -- 0
                                xAE
141                         after_state <= ResetOn;
142                         current_state <= Transition1;
143                     when ResetOn =>
144                         temp_res <= '0';
145                         current_state <= Wait2;
146                     when Wait2 =>
147                         after_state <= ResetOff;
148                         current_state <= Transition3;
149                     when ResetOff =>
150                         temp_res <= '1';
151                         after_state <= ChargePump1;
152                         current_state <= Transition3;
153                     when ChargePump1 =>
154                         temp_sdata <= "10001101"; -- 0
                                x8D
155                         after_state <= ChargePump2;
156                         current_state <= Transition1;
157                     when ChargePump2 =>
158                         temp_sdata <= "00010100"; -- 0
                                x14
159                         after_state <= PreCharge1;
160                         current_state <= Transition1;
161                     when PreCharge1  =>
162                         temp_sdata <= "11011001"; -- 0
                                xD9
163                         after_state <= PreCharge2;
164                         current_state <= Transition1;
165                     when PreCharge2 =>
166                         temp_sdata <= "11110001"; -- 0
                                xF1
```

```
167                         after_state <= VbatOn;
168                         current_state <= Transition1;
169                     when VbatOn =>
170                         temp_vbat <= '0';
171                         current_state <= Wait3;
172                     when Wait3 =>
173                         after_state <= DispContrast1;
174                         current_state <= Transition3;
175                     when DispContrast1=>
176                         temp_sdata <= "10000001"; -- 0
                               x81
177                         after_state <= DispContrast2;
178                         current_state <= Transition1;
179                     when DispContrast2=>
180                         temp_sdata <= "00001111"; -- 0
                               x0F
181                         after_state <= InvertDisp1;
182                         current_state <= Transition1;
183                     when InvertDisp1 =>
184                         temp_sdata <= "10100000"; -- 0
                               xA0
185                         after_state <= InvertDisp2;
186                         current_state <= Transition1;
187                     when InvertDisp2 =>
188                         temp_sdata <= "11000000"; -- 0
                               xC0
189                         after_state <= ComConfig1;
190                         current_state <= Transition1;
191                     when ComConfig1 =>
192                         temp_sdata <= "11011010"; -- 0
                               xDA
193                         after_state <= ComConfig2;
194                         current_state <= Transition1;
195                     when ComConfig2 =>
196                         temp_sdata <= "00000000"; -- 0
                               x00
197                         after_state <= DispOn;
198                         current_state <= Transition1;
199                     when DispOn =>
200                         temp_sdata <= "10101111"; -- 0
                               xAF
201                         after_state <= Done;
202                         current_state <= Transition1;
```

```
203                         -- End Initialization sequence
204
205                         -- Used for debugging, turns the
                               entire screen on regardless of
                               memory
206                         when FullDisp =>
207                             temp_sdata <= "10100101"; -- 0
                                   xA5
208                             after_state <= Done;
209                             current_state <= Transition1;
210
211                         -- Done state
212                         when Done =>
213                             if en = '0' then
214                                 temp_fin <= '0';
215                                 current_state <= Idle;
216                             else
217                                 temp_fin <= '1';
218                             end if;
219
220                         -- SPI transitions
221                         -- 1. Set en to 1
222                         -- 2. Waits for spi_ctrl to finish
223                         -- 3. Goes to clear state (
                               Transition5)
224                         when Transition1 =>
225                             temp_spi_en <= '1';
226                             current_state <= Transition2;
227                         when Transition2 =>
228                             if temp_spi_fin = '1' then
229                                 current_state <= Transition5
                                       ;
230                             end if;
231                         -- End SPI transitions
232
233                         -- Delay transitions
234                         -- 1. Set delay_en to 1
235                         -- 2. Waits for delay to finish
236                         -- 3. Goes to Clear state (
                               Transition5)
237                         when Transition3 =>
238                             temp_delay_en <= '1';
239                             current_state <= Transition4;
```

```
240                         when Transition4 =>
241                             if temp_delay_fin = '1' then
242                                 current_state <= Transition5
                                        ;
243                             end if;
244                         -- End delay transitions
245
246                         -- Clear transitions
247                         -- 1. Sets both delay_en and en to 0
248                         -- 2. Go to after state
249                         when Transition5 =>
250                             temp_spi_en <= '0';
251                             temp_delay_en <= '0';
252                             current_state <= after_state;
253                         -- End Clear transitions
254
255                         when others =>
256                             current_state <= Idle;
257                     end case;
258                 end if;
259             end if;
260         end process;
261
262 end behavioral;
```

Listing 4: oled_init.vhd

Lectura guiada y criterios de integración

- **Propósito general**. Se implementa una FSM que ejecuta la secuencia de arranque típica de un controlador tipo SSD1306: `DISPLAY_OFF (0xAE)` → pulso de `RESET` → `CHARGE_PUMP (0x8D,0x14)` → `PRECHARGE (0xD9,0xF1)` → habilitación de `VBAT` → contraste → mapeos → `DISPLAY_ON (0xAF)`.
- **Control de alimentación**. `oled_vdd` y `oled_vbat` se manejan como señales activas en nivel bajo (`'0'` = habilitado). Este mapeo debe corresponder al hardware real; en caso contrario, se debe invertir la lógica.
- **Delays críticos**. La constante `temp_delay_ms` introduce 100ms únicamente cuando `after_state=DispContrast1` (comentado como "después de `VbatOn`"). En el resto de transiciones se emplea 1ms. Si la hoja

de datos exige tiempos mayores para carga de `VBAT` o post-`RESET`, se deben aumentar dichos márgenes.

- **Modo comando/datos**. `oled_dc` se fuerza a '0' al arrancar (modo comando). No se cambia a modo datos en esta FSM, lo cual es coherente al tratarse únicamente de inicialización.
- **Transferencias SPI**. Las macro-estados *Transition1/2/5* encapsulan el envío de un byte por `spi_ctrl`: *Transition1* habilita el envío; *Transition2* espera `temp_spi_fin`; *Transition5* limpia `temp_spi_en` y avanza.
- **Delays programados**. Las macro-estados *Transition3/4/5* encapsulan la espera temporizada mediante `delay.vhd` con `delay_en` y `delay_fin`.
- **Ajustes de mapeo**. Los estados etiquetados como *InvertDisp1/2* envían `0xA0` (SEG remap normal) y `0xC0` (COM scan normal); esto *no* invierte la imagen, solo fija el sentido de barrido. La etiqueta puede inducir a confusión; se sugiere renombrar a *SegRemap/ComScan*.
- **Contraste**. `0x81, 0x0F` fija un contraste bastante bajo. Si la luminancia resulta insuficiente, se sugiere aumentar a `0x7F` o el valor recomendado en la hoja de datos del panel concreto.
- **Fin de secuencia**. La bandera `fin` permanece alta en *Done* mientras `en` se mantenga en '1'. Para rearmar, se debe llevar `en` a '0'.

4.5. Demo de escritura en la OLED (oled$_ex.vhd$)

Código fuente

```
1  -- Description: Demo for the OLED display. First
       displays the alphabet for ~4 seconds and then
2  -- clears the display, waits for a ~1 second and then
       displays "Hello world!".
3
4  library ieee;
5  use ieee.std_logic_1164.all;
6  use ieee.std_logic_arith.all;
7  use ieee.std_logic_unsigned.all;
8
9  entity oled_ex is
10     port (  clk         : in std_logic; -- System clock
```

```
11              rst          : in std_logic; -- Global
                   synchronous reset
12              en           : in std_logic; -- Block enable
                   pin
13              sdout        : out std_logic; -- SPI data out
14              oled_sclk    : out std_logic; -- SPI clock
15              oled_dc      : out std_logic; -- Data/Command
                    controller
16              fin          : out std_logic); -- Finish flag
                    for block
17  end oled_ex;
18
19  architecture behavioral of oled_ex is
20
21      -- SPI controller
22      component spi_ctrl
23          port (  clk          : in std_logic;
24                  rst          : in std_logic;
25                  en           : in std_logic;
26                  sdata        : in std_logic_vector (7
                       downto 0);
27                  sdout        : out std_logic;
28                  oled_sclk    : out std_logic;
29                  fin          : out std_logic);
30      end component;
31
32      -- delay controller
33      component delay
34          port (  clk          : in std_logic;
35                  rst          : in std_logic;
36                  delay_ms     : in std_logic_vector (11
                       downto 0);
37                  delay_en     : in std_logic;
38                  delay_fin    : out std_logic);
39      end component;
40
41      -- character library, latency = 1
42      component ascii_rom
43          port (  clk     : in std_logic; -- System clock
44                  addr    : in std_logic_vector (10 downto
                       0); -- First 8 bits is the ASCII
                       value of the character, the last 3
                       bits are the parts of the char
```

```
45                  dout    : out std_logic_vector (7 downto
                       0)); -- Data byte out
46      end component;
47
48      -- States for state machine
49      type states is (Idle,
50                      ClearDC,
51                      SetPage,
52                      PageNum,
53                      LeftColumn1,
54                      LeftColumn2,
55                      SetDC,
56                      Alphabet,
57                      Wait1,
58                      ClearScreen,
59                      Wait2,
60                      HelloWorldScreen,
61                      UpdateScreen,
62                      SendChar1,
63                      SendChar2,
64                      SendChar3,
65                      SendChar4,
66                      SendChar5,
67                      SendChar6,
68                      SendChar7,
69                      SendChar8,
70                      ReadMem,
71                      ReadMem2,
72                      Done,
73                      Transition1,
74                      Transition2,
75                      Transition3,
76                      Transition4,
77                      Transition5);
78
79      type oled_mem is array (0 to 3, 0 to 15) of
           std_logic_vector (7 downto 0);
80
81      -- Variable that contains what the screen will be
           after the next UpdateScreen state
82      signal current_screen : oled_mem;
83      -- Constant that contains the screen filled with the
            Alphabet and numbers
```

```
84      constant alphabet_screen : oled_mem := ((x"41", x"42
            ", x"43", x"44", x"45", x"46", x"47", x"48", x"49
            ", x"4A", x"4B", x"4C", x"4D", x"4E", x"4F", x"50
            "),
85      (x"51", x"52", x"53", x"54", x"55", x"56", x"57", x"
            58", x"59", x"5A", x"61", x"62", x"63", x"64", x"
            65", x"66"),
86      (x"67", x"68", x"69", x"6A", x"6B", x"6C", x"6D", x"
            6E", x"6F", x"70", x"71", x"72", x"73", x"74", x"
            75", x"76"),
87      (x"77", x"78", x"79", x"7A", x"30", x"31", x"32", x"
            33", x"34", x"35", x"36", x"37", x"38", x"39", x"
            7F", x"7F"));
88
89      -- Constant that fills the screen with blank (spaces
            ) entries
90      constant clear_screen : oled_mem := (   (x"20", x"20
            ", x"20", x"20", x"20", x"20", x"20", x"20", x"20
            ", x"20", x"20", x"20", x"20", x"20", x"20", x"20
            "),
91      (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20"),
92      (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20"),
93      (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20"));
94
95      -- Constant that holds "Hello world!"
96      constant hello_world_screen : oled_mem := ( (x"48",
            x"65", x"6c", x"6c", x"6f", x"20", x"77", x"6f",
            x"72", x"6c", x"64", x"21", x"20", x"20", x"20",
            x"20"),
97      (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20"),
98      (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20"),
99      (x"20", x"20", x"20", x"20", x"20", x"20", x"20", x"
            20", x"20", x"20", x"20", x"20", x"20", x"20", x"
```

```
        20", x"20"));
100
101     -- Current overall state of the state machine
102     signal current_state : states := Idle;
103
104     -- State to go to after the SPI transmission is
           finished
105     signal after_state : states;
106
107     -- State to go to after the set page sequence
108     signal after_page_state : states;
109
110     -- State to go to after sending the character
           sequence
111     signal after_char_state : states;
112
113     -- State to go to after the UpdateScreen is finished
114     signal after_update_state : states;
115
116     -- Contains the value to be outputted to oled_dc
117     signal temp_dc : std_logic := '0';
118
119     -- Used in the Delay controller block
120     signal temp_delay_ms : std_logic_vector (11 downto
           0); -- Amount of ms to delay
121     signal temp_delay_en : std_logic := '0'; -- Enable
           signal for the Delay block
122     signal temp_delay_fin : std_logic; -- Finish signal
           for the Delay block
123
124     -- Used in the SPI controller block
125     signal temp_spi_en : std_logic := '0'; -- Enable
           signal for the SPI block
126     signal temp_sdata : std_logic_vector (7 downto 0) :=
            (others => '0'); -- Data to be sent out on SPI
127     signal temp_spi_fin : std_logic; -- Finish signal
           for the SPI block
128
129     signal temp_char : std_logic_vector (7 downto 0) :=
           (others => '0'); -- Contains ASCII value for
           character
130     signal temp_addr : std_logic_vector (10 downto 0) :=
            (others => '0'); -- Contains address to byte
```

```
        needed in memory
131     signal temp_dout : std_logic_vector (7 downto 0); --
            Contains byte outputted from memory
132     signal temp_page : std_logic_vector (1 downto 0) :=
            (others => '0'); -- Current page
133     signal temp_index : integer range 0 to 15 := 0; --
            Current character on page
134
135 begin
136
137     oled_dc <= temp_dc;
138
139     -- "Example" finish flag only high when in done
            state
140     fin <= '1' when current_state = Done else '0';
141
142     -- Instantiate SPI controller
143     spi_comp: spi_ctrl port map (   clk => clk,
144                                     rst => rst,
145                                     en => temp_spi_en,
146                                     sdata => temp_sdata,
147                                     sdout => sdout,
148                                     oled_sclk =>
                                            oled_sclk,
149                                     fin => temp_spi_fin)
                                            ;
150
151     -- Instantiate delay
152     delay_comp: delay port map (clk => clk,
153                                 rst => rst,
154                                 delay_ms =>
                                        temp_delay_ms,
155                                 delay_en =>
                                        temp_delay_en,
156                                 delay_fin =>
                                        temp_delay_fin);
157
158     -- Instantiate ASCII character library
159     char_lib_comp : ascii_rom port map (clk => clk,
160                                         addr =>
                                                temp_addr,
161                                         dout =>
                                                temp_dout);
```

```
162
163     process (clk)
164     begin
165         if rising_edge(clk) then
166             case current_state is
167                 -- Idle until en pulled high than
                       intialize Page to 0 and go to state
                       alphabet afterwards
168                 when Idle =>
169                     if en = '1' then
170                         current_state <= ClearDC;
171                         after_page_state <= Alphabet;
172                         temp_page <= "00";
173                     end if;
174                 -- Set current_screen to constant
                       alphabet_screen and update the screen
                       ; go to state Wait1 afterwards
175                 when Alphabet =>
176                     current_screen <= alphabet_screen;
177                     current_state <= UpdateScreen;
178                     after_update_state <= Wait1;
179                 -- Wait 4ms and go to ClearScreen
180                 when Wait1 =>
181                     temp_delay_ms <= "111110100000"; --
                           4000
182                     after_state <= ClearScreen;
183                     current_state <= Transition3; --
                           Transition3 = delay transition
                           states
184                 -- Set current_screen to constant
                       clear_screen and update the screen;
                       go to state Wait2 afterwards
185                 when ClearScreen =>
186                     current_screen <= clear_screen;
187                     after_update_state <= Wait2;
188                     current_state <= UpdateScreen;
189                 -- Wait 1ms and go to HelloWorldScreen
190                 when Wait2 =>
191                     temp_delay_ms <= "001111101000"; --
                           1000
192                     after_state <= HelloWorldScreen;
193                     current_state <= Transition3; --
                           Transition3 = delay transition
```

```
                    states
194                 -- Set currentScreen to constant
                       hello_world_screen and update the
                       screen; go to state Done afterwards
195                 when HelloWorldScreen =>
196                     current_screen <= hello_world_screen
                           ;
197                     after_update_state <= Done;
198                     current_state <= UpdateScreen;
199                 -- Do nothing until en is deassertted
                       and then current_state is Idle
200                 when Done             =>
201                     if en = '0' then
202                         current_state <= Idle;
203                     end if;
204
205                 -- UpdateScreen State
206                 -- 1. Gets ASCII value from
                       current_screen at the current page
                       and the current spot
207                 --    of the page
208                 -- 2. If on the last character of the
                       page transition update the page
                       number, if on
209                 --    the last page(3) then the
                       updateScreen go to "
                       after_update_state" after
210                 when UpdateScreen =>
211                     temp_char <= current_screen(
                           conv_integer(temp_page),
                           temp_index);
212                     if temp_index = 15 then
213                         temp_index <= 0;
214                         temp_page <= temp_page + 1;
215                         after_char_state <= ClearDC;
216                         if temp_page = "11" then
217                             after_page_state <=
                                   after_update_state;
218                         else
219                             after_page_state <=
                                   UpdateScreen;
220                         end if;
221                     else
```

```
222                         temp_index <= temp_index + 1;
223                         after_char_state <= UpdateScreen
                               ;
224                     end if;
225                     current_state <= SendChar1;
226
227                 -- Update Page states
228                 -- 1. Sets oled_dc to command mode
229                 -- 2. Sends the SetPage Command
230                 -- 3. Sends the Page to be set to
231                 -- 4. Sets the start pixel to the left
                       column
232                 -- 5. Sets oled_dc to data mode
233                 when ClearDC =>
234                     temp_dc <= '0';
235                     current_state <= SetPage;
236                 when SetPage =>
237                     temp_sdata <= "00100010";
238                     after_state <= PageNum;
239                     current_state <= Transition1;
240                 when PageNum =>
241                     temp_sdata <= "000000" & temp_page;
242                     after_state <= LeftColumn1;
243                     current_state <= Transition1;
244                 when LeftColumn1 =>
245                     temp_sdata <= "00000000";
246                     after_state <= LeftColumn2;
247                     current_state <= Transition1;
248                 when LeftColumn2 =>
249                     temp_sdata <= "00010000";
250                     after_state <= SetDC;
251                     current_state <= Transition1;
252                 when SetDC =>
253                     temp_dc <= '1';
254                     current_state <= after_page_state;
255                 -- End update Page states
256
257                 -- Send character states
258                 -- 1. Sets the address to ASCII value of
                        character with the counter appended
                       to the
259                 --    end
```

```
260                     -- 2. Waits a clock cycle for the data
                           to get ready by going to ReadMem and
                           ReadMem2
261                     --    states
262                     -- 3. Send the byte of data given by the
                            ROM
263                     -- 4. Repeat 7 more times for the rest
                           of the character bytes
264                     when SendChar1 =>
265                         temp_addr <= temp_char & "000";
266                         after_state <= SendChar2;
267                         current_state <= ReadMem;
268                     when SendChar2 =>
269                         temp_addr <= temp_char & "001";
270                         after_state <= SendChar3;
271                         current_state <= ReadMem;
272                     when SendChar3 =>
273                         temp_addr <= temp_char & "010";
274                         after_state <= SendChar4;
275                         current_state <= ReadMem;
276                     when SendChar4 =>
277                         temp_addr <= temp_char & "011";
278                         after_state <= SendChar5;
279                         current_state <= ReadMem;
280                     when SendChar5 =>
281                         temp_addr <= temp_char & "100";
282                         after_state <= SendChar6;
283                         current_state <= ReadMem;
284                     when SendChar6 =>
285                         temp_addr <= temp_char & "101";
286                         after_state <= SendChar7;
287                         current_state <= ReadMem;
288                     when SendChar7 =>
289                         temp_addr <= temp_char & "110";
290                         after_state <= SendChar8;
291                         current_state <= ReadMem;
292                     when SendChar8 =>
293                         temp_addr <= temp_char & "111";
294                         after_state <= after_char_state;
295                         current_state <= ReadMem;
296                     when ReadMem =>
297                         current_state <= ReadMem2;
298                     when ReadMem2 =>
```

```
299                     temp_sdata <= temp_dout;
300                     current_state <= Transition1;
301                 -- End send character states
302 
303                 -- SPI transitions
304                 -- 1. Set en to 1
305                 -- 2. Waits for spi_ctrl to finish
306                 -- 3. Goes to clear state (Transition5)
307                 when Transition1 =>
308                     temp_spi_en <= '1';
309                     current_state <= Transition2;
310                 when Transition2 =>
311                     if temp_spi_fin = '1' then
312                         current_state <= Transition5;
313                     end if;
314                 -- End SPI transitions
315 
316                 -- Delay transitions
317                 -- 1. Set delay_en to 1
318                 -- 2. Waits for delay to finish
319                 -- 3. Goes to Clear state (Transition5)
320                 when Transition3 =>
321                     temp_delay_en <= '1';
322                     current_state <= Transition4;
323                 when Transition4 =>
324                     if temp_delay_fin = '1' then
325                         current_state <= Transition5;
326                     end if;
327                 -- End Delay transitions
328 
329                 -- Clear transition
330                 -- 1. Sets both delay_en and en to 0
331                 -- 2. Go to after state
332                 when Transition5 =>
333                     temp_spi_en <= '0';
334                     temp_delay_en <= '0';
335                     current_state <= after_state;
336                 -- End Clear transition
337 
338                 when others =>
339                     current_state <= Idle;
340             end case;
341         end if;
```

```
342     end process;
343
344 end behavioral;
```

Listing 5: oled_ex.vhd

Lectura guiada y criterios de integración

- **Flujo general**. La FSM muestra *Alphabet* durante $m4$ s, limpia la pantalla 1 s y luego escribe *Hello world!*. Cada "pantalla" es una matriz `oled_mem`$[4 \times 16]$ de códigos ASCII; 4 páginas (32 px altos / 8 px por página) y 16 columnas de caracteres (16×8 px $= 128$ px de ancho).
- **Direccionamiento de página/columna**. La secuencia texttttClearDC `SetPage PageNum LeftColumn1/2 SetDC` sitúa el cursor al inicio de cada línea. Se usa `0x22` seguido de número de página y luego nibble bajo/alto de columna (`0x00`/`0x10`). En algunos controladores compatibles se espera `0xB0|page`; si el panel no responde, se debe revisar este punto contra su *datasheet*.
- **Lectura de fuente y latencia**. `ascii_rom` tiene salida registrada (latencia 1 ciclo). Por ello se intercalan `ReadMem` → `ReadMem2` antes de transmitir cada byte de glifo.
- **Rendimiento estimado**. Con `spi_ctrl` a `clk`/32 y `clk` = 100 MHz, $f_{\text{SPI}} \approx 3{,}125$ MHz. Un fotograma completo envía $4 \times 16 \times 8 = 512$ bytes $\Rightarrow$ 4096 bits $\approx 1{,}31$ ms de tiempo útil de enlace (sin contar *overhead*); margen amplio para $4s/1s$ de espera.
- **Control de `DC` y modo**. `oled_dc`=0 en comandos de posicionamiento; `oled_dc`=1 al enviar datos de glifos. La señal `CS_n` (si existe en el hardware) debe ser orquestada en la capa superior, pues este archivo no la maneja.
- **Estado final**. `fin` es nivel alto únicamente en `Done`. Para rearmar, `en` debe caer a '0'.

4.6. Módulo tope del controlador OLED (oled ctrl.vhd)

Código fuente

```
--
-- Written by Ryan Kim, Digilent Inc.
-- Modified by Michael Mattioli
--
-- Description: Top level controller that controls the
   OLED display.
--

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity oled_ctrl is
    port (  clk         : in std_logic;
            rst         : in std_logic;
            oled_sdin   : out std_logic;
            oled_sclk   : out std_logic;
            oled_dc     : out std_logic;
            oled_res    : out std_logic;
            oled_vbat   : out std_logic;
            oled_vdd    : out std_logic);
end oled_ctrl;

architecture behavioral of oled_ctrl is

    component oled_init is
        port (  clk         : in std_logic;
                rst         : in std_logic;
                en          : in std_logic;
                sdout       : out std_logic;
                oled_sclk   : out std_logic;
                oled_dc     : out std_logic;
                oled_res    : out std_logic;
                oled_vbat   : out std_logic;
                oled_vdd    : out std_logic;
                fin         : out std_logic);
    end component;

    component oled_ex is
        port (  clk         : in std_logic;
                rst         : in std_logic;
```

```
42                  en          : in std_logic;
43                  sdout       : out std_logic;
44                  oled_sclk   : out std_logic;
45                  oled_dc     : out std_logic;
46                  fin         : out std_logic);
47      end component;
48
49      type states is (Idle, OledInitialize, OledExample,
           Done);
50
51      signal current_state : states := Idle;
52
53      signal init_en          : std_logic := '0';
54      signal init_done        : std_logic;
55      signal init_sdata       : std_logic;
56      signal init_spi_clk     : std_logic;
57      signal init_dc          : std_logic;
58
59      signal example_en       : std_logic := '0';
60      signal example_sdata    : std_logic;
61      signal example_spi_clk  : std_logic;
62      signal example_dc       : std_logic;
63      signal example_done     : std_logic;
64
65  begin
66
67      Initialize: oled_init port map (clk,
68                                      rst,
69                                      init_en,
70                                      init_sdata,
71                                      init_spi_clk,
72                                      init_dc,
73                                      oled_res,
74                                      oled_vbat,
75                                      oled_vdd,
76                                      init_done);
77
78      Example: oled_ex port map ( clk,
79                                  rst,
80                                  example_en,
81                                  example_sdata,
82                                  example_spi_clk,
83                                  example_dc,
```

```
84                                  example_done);
85
86      -- MUXes to indicate which outputs are routed out
           depending on which block is enabled
87      oled_sdin <= init_sdata when current_state =
           OledInitialize else example_sdata;
88      oled_sclk <= init_spi_clk when current_state =
           OledInitialize else example_spi_clk;
89      oled_dc <= init_dc when current_state =
           OledInitialize else example_dc;
90      -- End output MUXes
91
92      -- MUXes that enable blocks when in the proper
           states
93      init_en <= '1' when current_state = OledInitialize
           else '0';
94      example_en <= '1' when current_state = OledExample
           else '0';
95      -- End enable MUXes
96
97      process (clk)
98      begin
99          if rising_edge(clk) then
100             if rst = '1' then
101                 current_state <= Idle;
102             else
103                 case current_state is
104                     when Idle =>
105                         current_state <= OledInitialize;
106                     -- Go through the initialization
                           sequence
107                     when OledInitialize =>
108                         if init_done = '1' then
109                             current_state <= OledExample
                                   ;
110                         end if;
111                     -- Do example and do nothing when
                           finished
112                     when OledExample =>
113                         if example_done = '1' then
114                             current_state <= Done;
115                         end if;
116                     -- Do nthing
```

```
117                         when Done =>
118                             current_state <= Done;
119                         when others =>
120                             current_state <= Idle;
121                     end case;
122                 end if;
123             end if;
124         end process;
125 
126 end behavioral;
```

Listing 6: oledctrl.vhd

Lectura guiada y criterios de integración

- **Orquestación de alto nivel**. El módulo `oled_ctrl` secuencia dos bloques: `oled_init` (arranque y configuración del panel) y `oled_ex` (demo de escritura). La FSM de 4 estados pasa de *Idle* → *OledInitialize* → *OledExample* → *Done*.
- **Multiplexado de señales**. Mientras *OledInitialize* está activo, las salidas `oled_sdin/sclk/dc` se toman de `oled_init`; durante *OledExample*, se conmutan a las señales del bloque de ejemplo. Esta conmutación evita contención y garantiza que un único productor maneje el bus SPI en cada fase.
- **Habilitación por estado**. Las señales `init_en` y `example_en` se derivan directamente del estado. Cada submódulo expone `fin` (`init_done`/`example_done`) como condición de avance.
- **Alimentación y reset del panel**. `oled_init` además gobierna `oled_res`, `oled_vbat` y `oled_vdd` según la secuencia definida; el tope las expone directamente a pines externos.
- **Ausencia de `CS_n`**. El diseño no incluye chip select dedicado en el tope; si el hardware del panel exige `CS_n`, se debe añadir una línea de control en `spi_ctrl`/`oled_init`/`oled_ex` y enrutarla aquí.
- **Detención segura**. En *Done* el estado se mantiene estable; las salidas quedan en el origen del último bloque (demo). Si se requiere reiniciar la demo, se debe forzar `rst` o introducir una transición explícita adicional.

4.7. Asignación de pines para la OLED y señales de sistema

Mapa de señales (paquete)

Señal	Pin del encapsulado	I/O Standard
`clk`	Y9	LVCMOS18
`rst`	P16	LVCMOS33
`oled_dc`	U10	LVCMOS33
`oled_res`	U9	LVCMOS33
`oled_sclk`	AB12	LVCMOS33
`oled_sdin`	AA12	LVCMOS33
`oled_vbat`	U11	LVCMOS33
`oled_vdd`	U12	LVCMOS33

Parámetros temporales del reloj

- Periodo de `clk`: 100MHz (10ns), ciclo 50 %.

Criterios eléctricos y de integración

- Dominios de tensión: señales en el dominio de 1.8V deben declararse como `LVCMOS18`; señales en el dominio de 3.3V como `LVCMOS33`. En este diseño, `clk` opera a 1.8V; resto de señales (OLED y `rst`) operan a 3.3V.
- Polaridades de control: `oled_res`, `oled_vbat` y `oled_vdd` se manejan como *activos en bajo* en la lógica de inicialización; se debe confirmar su sentido activo real en el hardware y ajustar si procede.
- Señales SPI: `oled_sclk` y `oled_sdin` comparten dominio de 3.3V. La máxima frecuencia efectiva viene dada por el divisor interno del bloque SPI ($\approx$ 3.125MHz con 100MHz de sistema).